# 实验 5：大模型 Agent 部署训练实验指导

## 一、实验目标

1. 基于实验 1-4 的全流程成果，掌握大模型特别是 DeepSeek、ChatGLM、Qwen 等国产大模型 Agent 的核心开发流程，形成“数据-微调-框架集成-部署交互”的完整闭环；

2. 完成大模型环境部署与微调数据准备，复用实验 1 的文本数据与实验 4 的前后端数据规范，实现数据格式对齐；

3. 掌握基座模型选择与参数高效微调方法，结合实验 2 的 LoRA 技术与实验 3 的量化策略，在有限资源下实现大模型适配特定任务；

4. 构建大模型 Agent 核心逻辑，集成实验 4 的后端 API 作为工具，实现 Agent 的工具调用与任务拆解能力；

5. 完成 Agent 服务部署与前端交互集成，延续实验 4 的 API 规范与数据流设计，解决跨平台兼容性与分布式协调问题，强化工程化落地思维。

## 二、实验环境

硬件：

- 基础配置：CPU ≥ 16 核，内存 ≥ 64GB，GPU；
- 分布式支持：可选多 GPU 节点（用于大模型微调加速）。

软件：

- 大模型框架：PyCharm（Python 3.12+）、Anaconda（Python 3.12），**PyTorch ≥2.5**，**Transformers ≥4.40**，Accelerate 0.27+，PEFT 0.10+；
- 模型工具：使用 OpenAI Python SDK（DeepSeek 适配）、modelscope（魔搭社区模型）、zai SDK 或 OpenAI 兼容接口（ChatGLM 接口）、Ollama（本地部署）；
- Agent 框架：LangChain>=1.0
- 部署工具：**FastAPI ≥0.130**（延续实验 4），Docker 24.0+，Redis 7.2+（任务队列）；
- 前端工具：React 18+（复用实验 4 项目），Ant Design Pro（增强交互组件）；
- 关联依赖：实验 1 的文本数据集、实验 2 的 LoRA 微调代码、实验 3 的模型量化工具、实验 4 的后端 API 服务。

## 三、实验学时与安排

共 6 学时，分 4 个阶段完成：

- 阶段 1：环境与数据准备（1 学时）部署大模型环境，基于实验 1 数据构建微调数据集；
- 阶段 2：模型选择与微调（1 学时）用实验 2 的参数高效微调技术适配任务；
- 阶段 3：Agent 框架集成（2 学时）基于实验 4 API 构建 Agent 工具调用能力；
- 阶段 4：部署与测试（2 学时）集成到实验 4 前端，验证端到端交互。

## 四、实验步骤

### 阶段 1：环境与数据准备（5.1 模块）

目标：部署大模型训练与推理环境，基于实验 1 数据构建符合 Agent 微调需求的指令数据集，确保数据格式与实验 4 后端兼容。

#### 任务 5.1.1 大模型环境部署

1. 基础依赖安装

2. 大模型与量化配置：

- 下载开源基座模型（如 DeepSeek、**GLM-5/GLM-5.3** 等）
- 用实验 3 的量化方法加载模型

#### 任务 5.1.2 微调数据准备（关联实验 1/2/4）

1. 数据格式设计：基于实验 1 的文本分类任务（技术博客主题分类），构建“指令-输入-输出”格式的微调数据

2. 数据生成与清洗：

- 从实验 1 的 text_data.json 中抽取 100 条文本，用上述格式生成微调数据集 agent_finetune_data.json；
- 复用实验 1 的文本清洗逻辑（去停用词、标点标准化），确保输入文本一致性

### 阶段 2：模型选择与微调（5.2 模块）

目标：选择适配的基座模型（如 DeepSeek、ChatGLM 等），用实验 2 的参数高效微调技术（LoRA）在自定义数据集上微调，结合实验 3 的量化策略控制资源消耗。

#### 任务 5.2.1 基座模型选择与评估

1. 模型选择依据：

- 资源适配：根据实验硬件从当前主流开源模型中选型；

- 任务适配：优先选择支持中文/多轮对话的模型，适配实验 1 的中文文本（若实验 1 为英文则选通用模型）。

**DeepSeek-V4-Flash 或 DeepSeek-V4.1-Flash、GLM-5 或 GLM-5.3、Qwen3.5 系列**

2. 基线性能评估：

- 用实验 1 的 10 条未参与微调的文本测试基座模型零样本分类准确率

**任务 5.2.2 参数高效微调（LoRA）实践（关联实验 2）**

1. 微调配置：复用实验 2 的 LoRA 参数，适配大模型结构

2. 微调训练：用 SFTTrainer 训练，数据加载实验 1 转化的指令集

3. 微调效果评估：用实验 1 的测试集对比微调前后准确率

**阶段 3：Agent 框架集成（5.3 模块）**

目标：基于 LangChain 构建大模型 Agent 核心逻辑，集成实验 4 的后端 API 作为工具，实现任务拆解与工具调用能力。

**任务 5.3.1 Agent 核心逻辑设计**

1. 工具定义（关联实验 4 API）：将实验 4 的图像分类、文本分类 API 封装为 Agent 可调用的工具

2. 使用 create_agent 初始化 Agent

**任务 5.3.2 Agent 推理测试（关联实验 1 数据）**

1. 单轮任务测试：输入实验 1 的图像/文本，验证 Agent 工具调用准确性

2. 多轮任务测试：验证 Agent 记忆与上下文理解能力

**阶段 4：部署与测试（5.4 模块）**

目标：将大模型 Agent 服务部署为 API，集成到实验 4 的前端界面，验证端到端交互与跨平台兼容性。

**任务 5.4.1 基于 FastAPI 直接暴露**

1. Agent 服务封装为 API

**任务 5.4.2 前端交互集成（复用实验 4 项目）**

1. Agent 对话界面开发：在实验 4 的 React 项目中新增 AgentChat.tsx 组件

**任务 5.4.3 端到端测试与优化**

1. 功能测试场景：

- 场景 1：调用图像工具→ 输入实验 1 的猫图像路径→ 验证 Agent 返回“猫”类别；

- 场景 2：文本分类→ 输入实验 1 的技术文本→ 验证返回正确主题；

- 场景 3：多轮对话→ 先询问“如何判断图像类别”，再提供路径→ 验证 Agent 理解上下文。

2. 性能与兼容性优化：

- 推理延迟优化：用实验 3 的模型量化（4-bit）降低 Agent 响应时间（目标≤5 秒）；

- 跨平台测试：在 Chrome、Firefox、移动端浏览器验证界面适配与功能稳定性（复用实验 4 的响应式设计）；

- 异常处理：模拟 API 超时→ 前端显示“请重试”提示，后端记录错误日志。

**五、重点与难点说明**

重点关注

1. 前后端数据流设计：

- 延续实验 4 的 JSON 格式规范，Agent 的 query-response 与工具调用结果采用统一字段（user/agent/tool_calls），确保数据流转无歧义；

- 对话历史存储格式与实验 4 的用户会话逻辑对齐，支持断点续聊。

2. API 设计规范：

- Agent API 路径（/api/v1/agent/chat）与实验 4 的 API 版本（v1）保持一致，便于前端统一管理；

- 错误码复用实验 4 的设计（400 参数错、500 服务器错），新增 Agent 特有错误（如 404 ToolNotFound 工具未找到）。

3. 异常处理：

- 前端：捕获 Agent 推理超时、工具调用失败等错误，显示用户友好提示（如“图像分析工具暂时不可用，请稍后再试”）；

- 后端：实现工具调用重试机制（3 次重试），记录详细日志（含用户查询、工具响应）便于调试。

难点突破

1. 跨平台兼容性：

- 前端：用 CSS Flexbox+媒体查询适配移动端，限制 Agent 响应文本长度（自动换行），避免界面溢出；

- 后端：API 返回数据避免特殊字符（如 JSON 转义），跨域配置同时允许实验 4 前端（localhost:3000）与 Agent 服务（localhost:8001）。

2. 分布式任务协调：

- 工具调用队列：用 Redis+Celery 管理并发工具调用（如同时处理图像和文本任务），避免资源竞争；

- 大模型推理调度：对高并发请求，实现请求排队与优先级机制（实验 1 的关键任务优先）。

3. 训练配置与前端交互：

- 微调资源控制：用实验 3 的量化（4-bit）+ 梯度累积（8 步）降低显存占用，避免 OOM 错误；

- 交互体验优化：前端展示 Agent 思考过程（如“正在调用图像分类工具...”），减少用户等待焦虑。

**六、实验报告要求**

1. 说明小组成员构成，明确每个成员完成的任务；

2. 全流程关联分析：

- 绘制“实验 1 数据→实验 2 微调→实验 3 优化→实验 4 API→实验 5 Agent”的闭环架构图，标注关键数据流节点；

- 说明各阶段技术复用点（如实验 2 LoRA→大模型微调，实验 4 API→Agent 工具）。

3. Agent 性能评估：

- 工具调用准确率：统计 10 轮测试中 Agent 正确选择工具的比例（目标≥90%）；

- 响应延迟：记录不同任务（文本/图像）的平均响应时间，分析优化空间（如量化前后对比）。

4. 问题与解决方案：

- 记录大模型微调显存不足、Agent 工具调用失败、前端跨域等典型问题及解决

方法；

- 对比实验 2 的小模型微调与大模型微调的差异，总结参数高效微调的关键优化点。

## 七、参考资料


1. DeepSeek 开发者文档

2. **智谱 AI GLM-5 / GLM-5.3 API 文档（或 zai SDK 官方文档）**

3. 魔搭社区 Qwen 部署指南

4. LangChain Agent 开发指南

5. https://huggingface.co/models

6. https://ollama.com/search